

VK EDUCATION · КЕЙС «ПОИСКОВИК, RAG»

Гибридный поисковик по arXiv с RAG-суммаризацией

BM25 + векторный поиск (Qdrant/HNSW) поверх 7800 статей arXiv,
объединённые через Reciprocal Rank Fusion, с ответами LLM по источникам

Автор проекта · Дата защиты

Задача по ТЗ (MVP кейса)

Четыре обязательных пункта минимального жизнеспособного продукта

1

Парсинг информационного ресурса

Сбор данных из внешнего открытого источника

2

Загрузка данных в поисковые индексы

Построение структур для быстрого поиска

3

Сервис: запрос → поисковая выдача

API-эндпоинт с ранжированным результатом

4

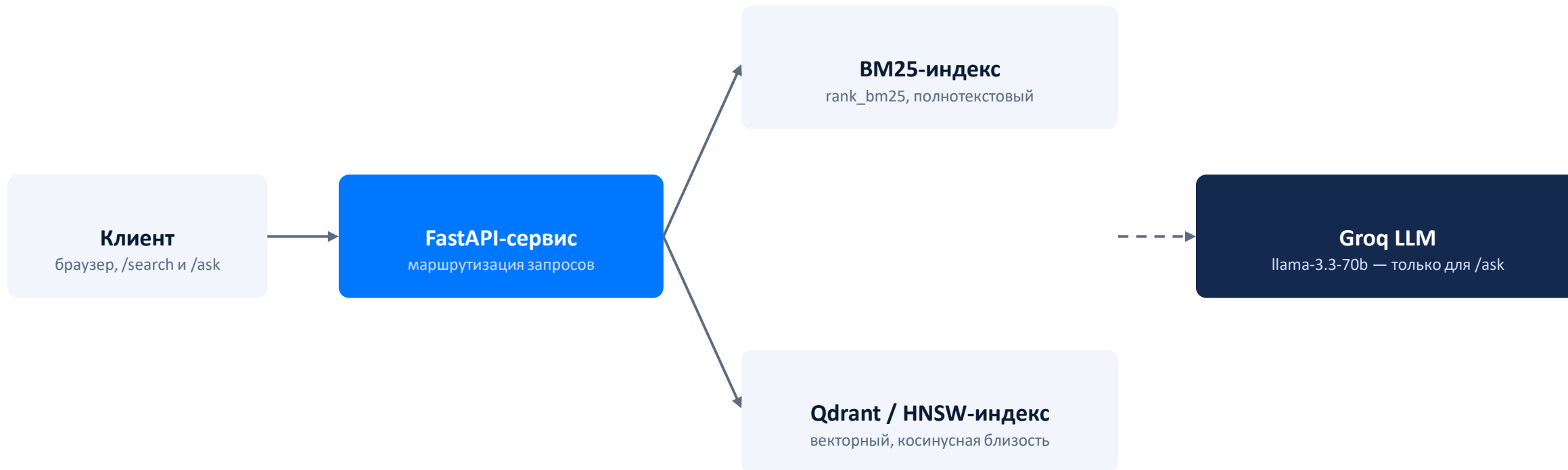
Суммаризация результата с помощью LLM

RAG-ответ на основе найденных документов

Все 4 пункта выполнены + расширения: гибридный поиск, ранжирование, локализация, UI

Архитектура решения

Два независимых индекса объединяются на уровне сервиса



BM25 и векторный индекс работают параллельно и независимо — сервис объединяет их выдачи через Reciprocal Rank Fusion (см. слайд «/search»).
Обращение к LLM происходит только на пути /ask.

MVP · 1. Парсинг информационного ресурса

- **Источник — arXiv API**

публичный, без API-ключа; категория cs.CL (Computation and Language / NLP)

- **Поля данных**

title, abstract, authors, категория, ссылка на статью

- **Формат хранения — JSONL**

дедупликация по id, соблюдение rate limit: пауза 3 сек между батчами по 100

- **Почему arXiv, а не скрейпинг сайта**

официальный API, нет риска бана, готовая структура данных, легко набрать 5000+ документов

7 800

документов собрано

- категория cs.CL
- JSONL, без дублей
- rate-limit safe

MVP · 2. Построение поисковых индексов

Строятся два индекса одновременно

BM25

- rank_bm25 — чистый Python
- работает полностью в памяти
- полнотекстовый поиск
- точные совпадения терминов

Векторный / HNSW

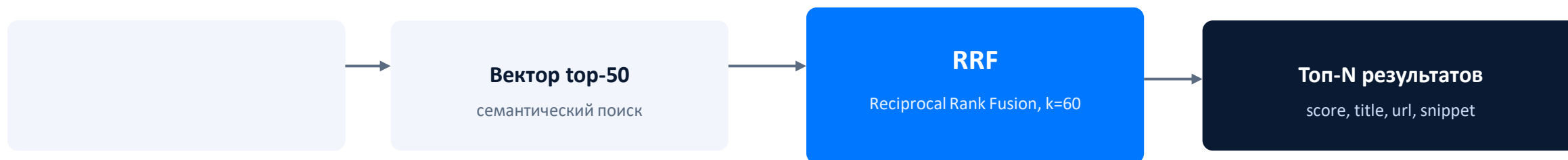
- Qdrant, embedded-режим (без сервера)
- эмбеддер BAAI/bge-small-en-v1.5
- поиск по косинусной близости
- хранит документ + метаданные + вектор

Один документ = один чанк (title + abstract) — дополнительный чанкинг не понадобился: тексты короткие

Почему Qdrant, а не голый FAISS: Qdrant — полноценная БД (документ + метаданные + вектор + персистентность); FAISS — только индекс без хранилища.

MVP · 3. Сервис поиска — /search

```
GET /search?q=...&top_n=N
```



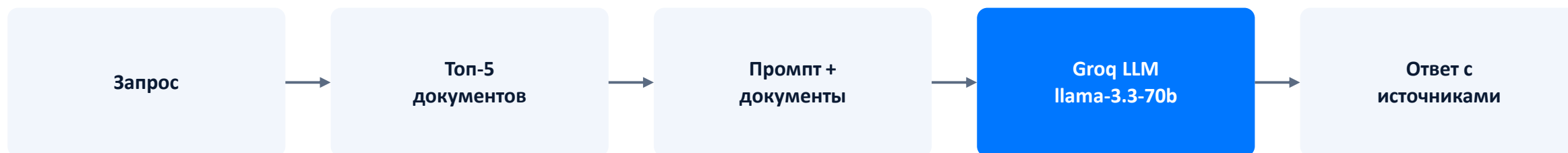
Ранжирования BM25 и вектора объединяются без нормализации score — RRF учитывает только позицию документа в каждой выдаче.

Ответ сервиса

Для каждого документа в топ-N: score, title, url, category, snippet.
На защите — живой запрос к /search и разбор ответа в браузере.

MVP · 4. RAG-суммаризация — /ask

```
GET /ask?q=...&top_n=5
```



Системный промпт

Отвечать только на основе найденных документов, не придумывать факты и явно говорить, если документы не отвечают на вопрос.

На защите — живой запрос к /ask: ответ модели и список документов-источников.

Расширение · поддержка русского языка

Проблема

Корпус документов и эмбеддер — англоязычные, а пользователи задают вопросы по-русски.

Решение

- Автоперевод запроса RU → EN перед поиском
- deep-translator / Google Translate, без API-ключа
- Системный промпт требует отвечать полностью на русском

До / после

Запрос на русском → корректный русскоязычный ответ с источниками — пример на защите.

Расширение · веб-интерфейс

Одностраничный UI без фреймворков, два режима работы

Режим: Поиск документов

hybrid retrieval arXiv

Title-ссылка на статью 1

score 0.90 · cs.CL · сниппет аннотации...

Title-ссылка на статью 2

score 0.91 · cs.CL · сниппет аннотации...

Title-ссылка на статью 3

score 0.92 · cs.CL · сниппет аннотации...

Режим: Спросить ИИ (RAG)

Ответ модели

Развёрнутый ответ LLM крупным текстом,
сформированный по найденным документам...

Источники

[1] arXiv:2310.xxxxx

[2] arXiv:2311.xxxxx

[3] arXiv:2312.xxxxx

[4] arXiv:2313.xxxxx

Технологический стек

Компонент	Технология
Полнотекстовый поиск	BM25 · rank_bm25
Векторный поиск	Qdrant · HNSW · cosine similarity
Эмбединги	BAAI/bge-small-en-v1.5
LLM	Groq · llama-3.3-70b-versatile
Перевод RU→EN	deep-translator
Backend	FastAPI + Uvicorn
Данные	arXiv API · 7 800 документов, cs.CL

Локальный запуск без облака

Демо можно записать и полностью прогнать на своей машине

1

Единственный тяжёлый шаг

Разовый подсчёт эмбедингов — пара минут на Apple Silicon (mps)

2

Всё остальное

Лёгкий CPU / IO — не требует GPU или долгих вычислений

3

LLM-инференс в облаке

Уходит в Groq API и не нагружает локальную машину

Демонстрация

Сценарии для живого показа

1

Поиск на английском языке

2

Поиск на русском языке (с автопереводом запроса)

3

RAG-ответ с указанием источников

4

Запрос без ответа в корпусе — модель честно отвечает «не знаю»

Итоги

Все пункты MVP выполнены

+ Гибридный поиск

+ Локализация (RU)

+ Веб-интерфейс

Что можно улучшить дальше

— Фильтрация выдачи по категориям

— Ранжирование с доп. сигналами

— Реляционный слой: пользователи, роли, история запросов

MVP полностью закрыт + 3 расширения сверх ТЗ

Вопросы

Репозиторий проекта

github.com/KrutoyarovSL/vk_praktika

Спасибо за внимание!